

Wazuh SIEM Deployment : First Detection Exercise

Author: Shubh Date: August 22-23, 2026 Lab Day: Day 1

Executive Summary

Deployed a Wazuh SIEM environment from scratch and onboarded one Windows and one Linux endpoint as monitored agents, with Sysmon installed on both for extended telemetry. Executed a controlled account-management exercise on the Windows endpoint (create → privilege change → delete) and validated end-to-end detection: raw Windows Security events were ingested, correctly ruled, and automatically mapped to MITRE ATT&CK techniques T1098, T1531, and T1484 in Wazuh’s Discover view.

Scope / Environment

Component	Details
SIEM Platform	Wazuh 4.14.7 (manager, indexer, dashboard – all-in-one)
SIEM Host	Ubuntu Server VM
Endpoint 1	Windows 10 Pro VM — Agent <code>WindowsVM</code> (ID 001)
Endpoint 2	Ubuntu 24.04 LTS VM — Agent <code>Shubh-Linux</code> (ID 002)
Hypervisor	VirtualBox
Telemetry	Sysmon deployed on both endpoints
Objective	Validate full pipeline: endpoint action → log → alert → MITRE mapping

1. Lab Topology

Three VMs were built for this lab, each serving a distinct role in the pipeline:

- Wazuh Manager (Ubuntu Server)** — runs the full Wazuh stack (manager, indexer, dashboard) and acts as the central log/alert aggregation point.
- Windows 10 Pro VM** — the primary monitored endpoint, used to generate Windows Security events for the detection exercise.
- Ubuntu 24.04 LTS VM** — a second monitored endpoint, kept separate from the manager host to simulate a realistic multi-machine environment rather than monitoring the manager itself.

All three VMs were networked together on an internal/host-only VirtualBox network so the agents could reach the manager’s IP directly.

2. Wazuh Server Deployment

Installed the Wazuh all-in-one stack (manager, indexer, dashboard) on the Ubuntu Server VM using the official install script:

```
curl -sO https://packages.wazuh.com/4.x/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
```

This single script provisions all three components together — the indexer (stores and indexes alert data), the manager (receives agent data, applies decoders/rules), and the dashboard (web UI for visualization and investigation).

Verification: Confirmed all three services were active and healthy before proceeding:

```
sudo systemctl status wazuh-manager wazuh-indexer wazuh-dashboard --no-pager
```

Logged into the dashboard at `https://<manager-ip>` and confirmed the Overview page showed indexer, manager, and dashboard all reporting healthy — this was the checkpoint before moving on to agent onboarding.

3. Endpoint Agent Onboarding

Windows Endpoint (`WindowsVM`):

1. Installed the Wazuh agent using the GUI MSI installer downloaded directly on the Windows VM (rather than a silent/CLI install), to keep the process transparent and verifiable at each step.
2. Since the agent installed without a pre-supplied authentication key, registered the agent manually from the manager side using the `manage_agents` utility:

```
sudo /var/ossec/bin/manage_agents
# A -> Add agent -> name: WindowsVM, IP: <windows-vm-ip>
# E -> Extract key -> agent ID 001
```

3. Pasted the extracted key into the **Wazuh Agent** GUI app on the Windows VM (Manager IP + Authentication Key fields), saved the configuration, then started the service via **Manage** → **Start**.

Result: Agent `WindowsVM` (ID 001) appeared as **Active** on the Endpoints dashboard.

Linux Endpoint (`Shubh-Linux`):

1. Deployed the agent using the dashboard's auto-generated Linux install command (DEB package), pointed at the manager's IP.
2. Registered using the same `manage_agents` workflow — add agent, extract key, import key on the endpoint side.

Result: Agent `Shubh-Linux` (ID 002) appeared as **Active** on the Endpoints dashboard.

Status after this stage: 2/2 agents active and reporting to the manager, confirmed on the Endpoints dashboard.

4. Sysmon Deployment

Sysmon was layered on top of both endpoints to capture telemetry beyond what default OS-level auditing provides — specifically process creation, account/permission changes, network connections, and file events.

- **Windows:** Installed with a standard Sysmon configuration from an elevated Administrator shell:

```
Sysmon.exe -i sysmonconfig.xml
```

- **Linux:** Installed Sysmon for Linux, confirmed the service was running, and validated it was actively generating events by tailing the systemd journal in real time:

```
systemctl status sysmon
sudo journalctl -u sysmon -f
```

Note: Linux Sysmon writes to the systemd journal rather than a native Windows-style Event Log. This distinction matters for how Wazuh needs to be configured to ingest Linux Sysmon data going forward (journal log collection — tracked as a follow-up item).

5. Detection Exercise: Account Creation → Privilege Change → Deletion

Scenario design: To generate a realistic, analyzable alert chain, a simple account-lifecycle sequence

was performed on `WindowsVM` – this pattern mirrors both routine admin activity and attacker persistence/cleanup behavior (create an account, elevate it, then remove evidence), making it a useful first exercise for practicing detection triage.

Actions performed, in order: 1. Created a new local user account (`student1`) 2. Added the account to the local Administrators group 3. Removed the account from the Administrators group 4. Deleted the user account entirely

Investigation process: After performing the actions, switched to Wazuh's **Discover** view on the dashboard and filtered by Windows Security Event ID to trace exactly which events were generated, how they were decoded, and what rules fired against them.

Findings

Windows Event ID	Description	Rule ID	Rule Level	MITRE ATT&CK Technique	MITRE Tactic(s)
4733	A member was removed from a security-enabled local group	60111	8	T1098 (Account Manipulation), T1531 (Account Access Removal)	Persistence, Impact
4726	A user account was deleted	60154	12	T1484 (Domain Policy Modification)	Defense Evasion, Privilege Escalation

Analysis

- **Severity escalation:** The rule level rose from 8 (group membership change) to 12 (account deletion, with mail alerting flagged true). This reflects Wazuh's rule engine treating account deletion as a materially higher-risk event than a permission change – deletion is harder to reverse and more consistent with cleanup/anti-forensic behavior.
- **Automatic MITRE mapping:** Wazuh's decoder/rule set attached technique and tactic metadata directly to each alert without any manual tagging on my part – this was derived automatically from the Windows Security event fields (`data.win.system.eventID` `data.win.eventdata.*`).
- **Audit trail integrity:** Each event captured the subject/target user, SID, domain, and logon ID, giving a complete “who did what to whom” record – enough to reconstruct the full sequence of actions purely from the alert data, without needing to reference the endpoint directly.

MITRE ATT&CK Reference

MITRE ATT&CK is a publicly maintained knowledge base of real-world adversary tactics and techniques, built from observed intrusions rather than theoretical models. It organizes attacker behavior into:

- **Tactics** – the attacker's goal (e.g., Persistence, Privilege Escalation, Defense Evasion)
- **Techniques** – the specific method used to achieve that goal (e.g., manipulating an account, modifying domain policy)

This framework is what lets an analyst turn a raw alert into an answer to “what stage of an attack does this represent, and what could plausibly come next” – which is exactly what this exercise practiced at small scale, using a self-generated (rather than simulated/canned) alert chain.

Key Takeaways

- Service state should always be verified directly (`systemctl status` , `Get-Service` , `dpkg -l`) rather than trusting an install script's summary output alone.
- MITRE ATT&CK mapping becomes far more concrete when tied to an actual triggered event – mapping T1098 to a real 4733 alert makes the framework click in a way that reading the matrix alone doesn't.
- A full detection pipeline (endpoint action → log → alert → framework mapping) can be validated end-to-end even in a small, two-endpoint lab environment, using the dashboard's Discover view as the primary investigation surface.

Next Steps (Day 2+)

- Fix Sysmon → Wazuh ingestion on the Linux agent (journald log collection)
- Explore File Integrity Monitoring (FIM) alerts
- Begin building toward a Kali VM for attacker-side telemetry generation

References

- MITRE ATT&CK Framework – <https://attack.mitre.org/>
- Wazuh Documentation – <https://documentation.wazuh.com/>